

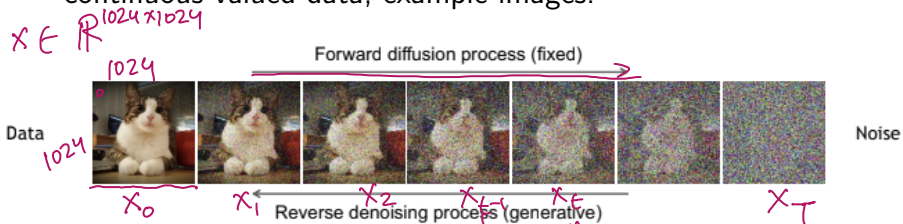
Denoising Diffusion Models (DDMs) for continuous data generation

Reading material

- 1 Denoising Diffusion Probabilistic Models (NeurIPS 2020) ✓
- 2 <https://cvpr2022-tutorial-diffusion-models.github.io/> ✓

Denoising Diffusion Models (DDMs)

- Applies the above structured variational objective for generating continuous valued data, example images.



Credits: <https://cvpr2022-tutorial-diffusion-models.github.io/>

Two steps during training:

- Forward process defines $q(\mathbf{x}_t | \mathbf{x}_{t-1})$: introduce random noise into the data in a series of steps until it turns into pure noise.
- Reverse or generation process defines $P_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$: starting with noise $P(\mathbf{x}_T)$ and gradually denoising it until it turns into data.

Forward process

During the forward process we start with an $\mathbf{x} \in D$, calling it step 0.

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbb{I}),$$
$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbb{I})$$

where $\beta_t \in (0, 1)$ is a noise schedule.

$$\mathbf{x}_1 = \sqrt{1 - \beta_1} \mathbf{x}_0 + \sqrt{\beta_1} \epsilon$$
$$\mathbf{x}_2 = \sqrt{1 - \beta_2} \mathbf{x}_1 + \sqrt{\beta_2} \epsilon$$

$$\mathbf{x}_T \rightarrow 0$$

Data



Noise

Reverse denoising process (generative)

Forward process (example)

$$\beta_t = 0.1t$$

- $x_0 = [2. \ -1. \ 3. \ 1. \ 1.5]$
- $x_1 = [1.48 \ -1.01 \ 2.87 \ 0.88 \ 1.75]$
- $x_2 = [1.29 \ -0.62 \ 3.19 \ 1.37 \ 1.47]$
- $x_3 = [0.99 \ -1.17 \ 2.52 \ 2. \ 1.06]$
- $x_4 = [-0.26 \ 0.4 \ 0.79 \ 0.77 \ 0.14]$
- $x_5 = [-0.63 \ 1.9 \ 0.83 \ 0.73 \ -0.67]$
- $x_6 = [-0.99 \ 0.04 \ 0.81 \ 1.35 \ -0.89]$
- $x_7 = [-0.08 \ -0.97 \ 0.02 \ 1.25 \ -0.36]$

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon \quad \epsilon \sim N(0, I)$$

<https://colab.research.google.com/drive/1yvq69JRu24QVRkkxb3WpghJMzE0CmXnT?usp=sharing>

Forward process: Direct from data

The forward process does not need to proceed sequentially because of the compositionality of Gaussian distribution.

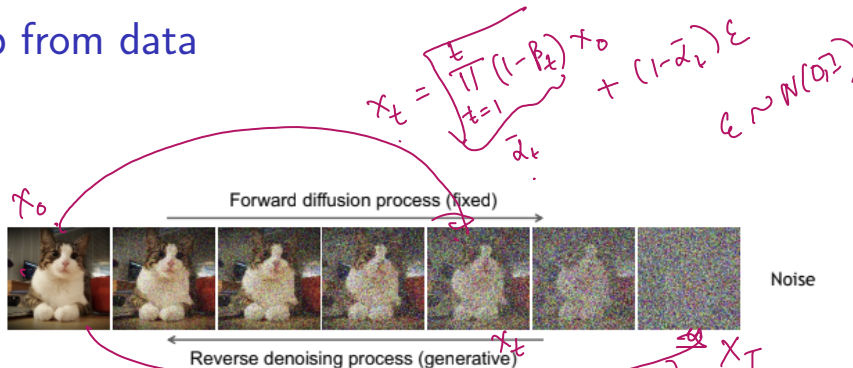
Iteratively apply the equation below (with $\epsilon \sim \mathcal{N}(0, \mathbb{I})$)

$$\begin{aligned} \mathbf{x}_t &= \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon \\ &= \sqrt{1 - \beta_t} (\sqrt{1 - \beta_{t-1}} \mathbf{x}_{t-2} + \sqrt{\beta_{t-1}} \epsilon) + \sqrt{\beta_t} \epsilon \\ &= \dots \\ \mathbf{x}_t &= \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon \end{aligned}$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$. That is,

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbb{I}),$$

Jump from data



The noise schedule β_t is designed such that $\bar{\alpha}_T \rightarrow 0$, ensuring $q(x_T | x_0) \approx \mathcal{N}(0, \mathbb{I})$.

Handwritten notes:

$$q(x_T | x_0)$$

$$\bar{\alpha}_t$$

Another example (1-d data)



Training objective

Recall, the structured variational objective.

$$\begin{aligned} \max_{\theta, q} \mathbb{E}_{\mathbf{x}_0 \sim D} \mathbb{E}_q [& \log P_{\theta}(\mathbf{x}_0 | \mathbf{x}_1) \\ & + KL(q(\mathbf{x}_T | \mathbf{x}_0); P(\mathbf{x}_T)) \quad (\text{holds by design}) \\ & + \sum_{t=1}^{T-1} KL(q(\mathbf{x}_t | \mathbf{x}_{t+1}, \mathbf{x}_0); P_{\theta}(\mathbf{x}_t | \mathbf{x}_{t+1}))] \end{aligned}$$

In the above,

- 1 The first term in the objective is assumed to be a Gaussian distribution with mean $\mathbf{x}_1$ and a fixed variance. So, nothing to learn. $P_{\theta}(\mathbf{x}_0 | \mathbf{x}_1) = \mathcal{N}(\mathbf{x}_0; \mathbf{x}_1, \sigma^2 \mathbb{I})$
- 2 The second term also has no parameters to be learned since by design we choose the noise schedule such that $q(\mathbf{x}_T | \mathbf{x}_0) = P(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; 0, \mathbb{I})$
- 3 We focus on learning only $P_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t)$, the denoiser via the last KL divergence. Let $L_{t-1} = KL(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0); P_{\theta}(\mathbf{x}_{t-1} | \mathbf{x}_t))$

Training the denoiser $P_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$!

First we need to calculate $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$.

$P(A|B,C) \propto P(B|A,C)P(C|A)$

In our case
 $P(B|A,C) = P(B|A)$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1}|\mathbf{x}_0)}{\int_{\mathbf{x}} q(\mathbf{x}_t|\mathbf{x}_{t-1})q(\mathbf{x}_{t-1}|\mathbf{x}_0)}$$

Substitute in the above:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_{t-1}}\mathbf{x}_{t-1}, \beta_{t-1}\mathbb{I})$$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \sqrt{\bar{\alpha}_{t-1}}\mathbf{x}_0, (1 - \bar{\alpha}_{t-1})\mathbb{I})$$

The above define a system of linear Gaussian equations, for which closed form solutions exist (https://www.seas.upenn.edu/~cis520/papers/Bishop_2.3.pdf)

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t\mathbb{I})$$

Training the denoiser $P_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$ II

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{1 - \beta_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t,$$
$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t.$$

Substitute $\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$:

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon \right).$$

Training objective

Parameterize μ_θ via a noise prediction network $\epsilon_\theta(\mathbf{x}_t, t)$:

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right).$$

The KL distance $KL(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0); P_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))$ becomes

$$L_{t-1} = \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2(1 - \beta_t)(1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] + C.$$

Thus, the KL distance objective reduces to a simple loss of minimizing the square error between the true noise and the predicted noise.

In practice ignoring these coefficients provides higher accuracy.

Training the Denoiser

Algorithm 1 Training Denoising Diffusion Model

0: **repeat**

0: $x_0 \sim q(x_0) \equiv \mathcal{D} \equiv \{x^1, x^2, \dots, x^N\}$

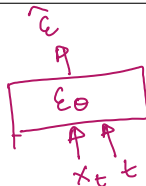
0: $t \sim \text{Uniform}(\{1, \dots, T\})$

0: $\epsilon \sim \mathcal{N}(0, \mathbb{I})$

0: Update θ via gradient descent on:

$$\underbrace{\nabla_{\theta} \|\underbrace{\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|}_{x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon}}^2$$

0: **until** converged = 0



Sampling

Algorithm 2 Sampling from Denoising Diffusion Model

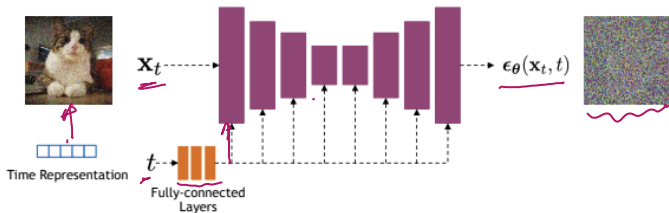
```
0:  $x_T \sim \mathcal{N}(0, \mathbb{I})$ 
0: for  $t = T, \dots, 1$  do
0:    $z \sim \mathcal{N}(0, \mathbb{I})$  if  $t > 1$ , else  $z = 0$  .
0:    $x_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \left( x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$ 
0: end for
0: return  $x_0 = 0$ 
```

Network Architecture

Implementation Considerations

Network Architectures

Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon_{\theta}(\mathbf{x}_t, t)$



Time representation: sinusoidal positional embeddings or random Fourier features.

Credits: <https://cvpr2022-tutorial-diffusion-models.github.io/>

Choosing noise schedule

Linear schedule: Vary beta-t within a small range gradually increasing, so we add more incremental noise in later time periods

Quadratic or cosine schedules

Learned noise schedule

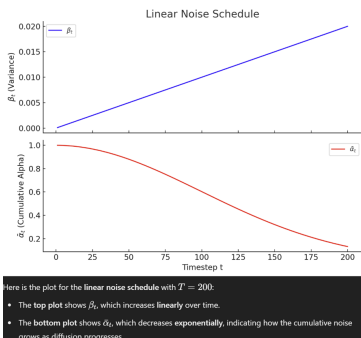


Figure: Noise schedule (Chat-gpt generated).

Link to code

https://colab.research.google.com/github/huggingface/diffusion-models-class/blob/main/unit1/02_diffusion_models_from_scratch.ipynb

Masked diffusion model for generating discrete data

Reading material

- 1 Simple and Effective Masked Diffusion Language Models
.Subham Sekhar Sahoo,Marianne Arriola,Yair Schiff,Aaron Gokaslan,Edgar Marroquin,Justin T Chiu,Alexander Rush,Volodymyr Kuleshov. NeurIPS 2024
<https://s-sahoo.com/mdlm/>
- 2 Simplified and Generalized Masked Diffusion for Discrete Data.
Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, Michalis K. Titsias. NeurIPS 2024

Motivation

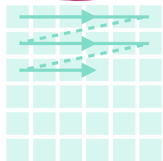
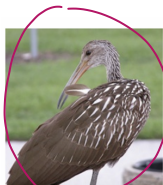
Useful for generating text, protein molecules from amino acid sequences, even images with discretized pixels

Default auto-regressive generation is slow, diffusion models allow for parallel generation of text. Examples

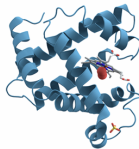
- 1 LLaDA: <https://ml-gsai.github.io/LLaDA-demo/>
- 2 Dream: <https://github.com/DreamLM/Dream>
- 3 Mercury coder: <https://www.inceptionlabs.ai/news>

Motivation

Fixed ordering not natural for some types of data¹.



HGFGTLEHPIYKVAKQWSMVHDTTVYFSCGLHVAAPATYVSMQAWKSTNDFPCRQHDNI
TMLYHINMESFVNLEFCNFQTDCKYLEDPWARHEKYP IRKAIKWEGLPNMQRLHMLHWIN
VSMDPNHGPVYCAKWDITILYMGKDGKERRTSAYMFTGVDEQHCERYEYRKFCGKHKAPKLM
GRLFRITKSCWWGCCTLDNMKPDKAKACAEDMRRCRNIPVVQNFQOCGKYWKATSQDNTK
RNSKCRAIEWEIFQYWINCSTVVKTFAPCMFGFQFRFHYGYNYMFVWTIKLSVYRWMPGV
DRETPVHAVNIINIWSAYKMTRYWCRIQCDSYWLWSGMTWRWCRWNREQPEWLSHDDMVQ
CWEGSYKLMFCGWRRHFISKSMVTLGGHKKDDGRRWMLQSTHHLHFPATINIHDDWDFPHG



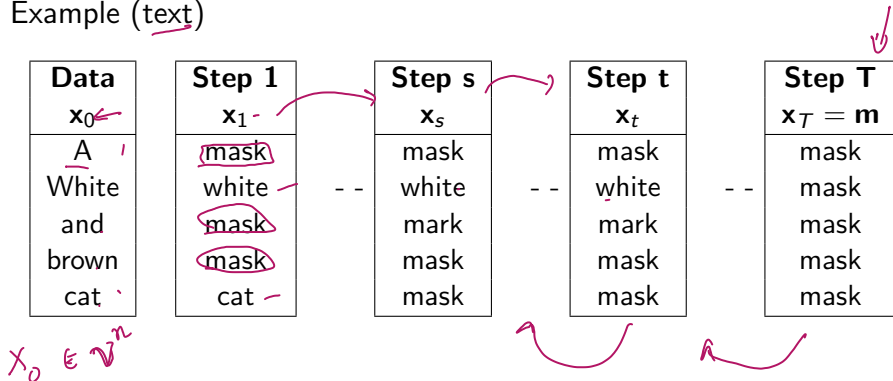
*Input
amino-acid
sequence*

*output 3-D
protein structure*

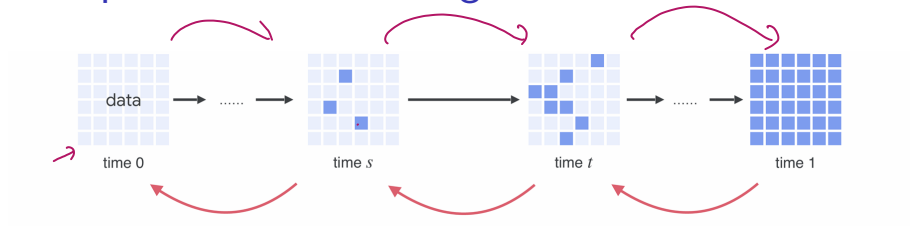
¹<https://neurips.cc/media/neurips-2024/Slides/93071.pdf>

Masked diffusion model for generating discrete data

Example (text)



Example: Discretized images



Forward process

Let $A \sim \text{Cat}(A; \pi)$ imply that A is discrete and follows a multinomial distribution with parameter π

During the forward process we start with an $\mathbf{x} \in D$, calling it step 0.

$$q(\mathbf{x}_t | \mathbf{x}_0) = \text{cat}(\mathbf{x}_t; \alpha_t \mathbf{x}_0 + (1 - \alpha_t) \mathbf{m})$$

where $1 - \alpha_t$ is the probability of masking. We choose $\alpha_t < \alpha_s$ for $s < t$.

Another way of writing the above (assume $n = 1$)

$$\begin{aligned} q(x_t | x_0) &= 1 - \alpha_t \quad \text{if } x_t = \mathbf{m} \\ &= \alpha_t \quad \text{if } x_t = x_0 \\ &= 0 \quad \text{otherwise.} \end{aligned}$$

Example to clarify notation

$$V \equiv \{v_1, v_2 \dots v_R\}$$

$n=1$ Generate one word.
 $v_R \equiv$ mask-token.
 R -dimensional.

$$x_0 \equiv 0/1 \text{ vector in}$$

$$\text{eg } x_0 = v_3$$

$$x_0 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$m = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}$$

$$\alpha_t x_0 + (1 - \alpha_t) m \equiv$$

$$\begin{bmatrix} 0 \\ 0 \\ \alpha_t \\ 0 \\ 0 \\ 1 - \alpha_t \end{bmatrix}$$

← sums to 1

we treat this vector as parameter
 of a multinomial distribution of R discrete
 values: which we denote as

$$x_t \sim \text{Cat}(x_t; \alpha_t x_0 + (1 - \alpha_t) m)$$

Example

Forward process

It can be shown that.

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \text{cat}(\mathbf{x}_t; \underbrace{\frac{\alpha_t}{\alpha_{t-1}} \mathbf{x}_{t-1}}_{< 1} + \underbrace{(1 - \frac{\alpha_t}{\alpha_{t-1}}) \mathbf{m}}_{\neq})$$

Handwritten notes:
 $\mathbf{x}_{t-1} \rightarrow \mathbf{x}_t$
 α_t are decreasing with t
 $\alpha_t < \alpha_{t-1}$

We choose α_t such that $\alpha_T \rightarrow 0$, so

$$\underline{q(\mathbf{x}_T | \mathbf{x}_0) \rightarrow 1} \quad \text{if} \quad \underline{\mathbf{x}_T = \mathbf{m}}$$